ENTERPRISE ANALYTICS HAKATHON REPORT

Executive Summary

MEHREEN FATIMA

Executive Summary

Modern enterprise databases are primarily designed to support transactional operations such as order processing, inventory management, customer management, and purchasing. While these databases efficiently handle day-to-day business operations, they are not optimized for analytical reporting because business metrics often require repeated joins, aggregations, and calculations across multiple tables.

The objective of this project was to design and implement a reusable analytics layer on top of an enterprise PostgreSQL database. Instead of querying raw operational tables repeatedly, analytical views were developed to centralize business logic and create reusable datasets for reporting. These views were then consumed in a Python-based analytics notebook to generate executive dashboards and business insights.

The solution follows the principles of Analytics Engineering by separating operational data from analytical reporting, ensuring scalability, maintainability, and improved query performance.

2. Project Objectives

The project was designed to achieve the following objectives:

- Build a reusable analytics layer over transactional data.
- Minimize repeated querying of operational tables.
- Create analytical views for multiple business domains.
- Develop a chained SQL pipeline for reusable reporting.
- Generate executive KPI datasets.
- Create business dashboards using Python.
- Produce actionable business recommendations.

3. Problem Statement

The operational AdventureWorks database contains highly normalized transactional tables optimized for data integrity and transactional processing. However, directly querying these tables for reporting introduces several challenges:

- Complex SQL joins
- Repeated calculations

- Poor maintainability
- Difficult dashboard development
- High query complexity

To address these issues, an intermediate analytics layer was developed that transforms raw operational data into reusable analytical datasets.

4. Project Overview

The project consists of two major components:

SQL Analytics Layer

Reusable analytical views were created to preprocess and organize business information.

Python Executive Dashboard

A Jupyter Notebook was developed to read only the analytical views and generate executive dashboards with business insights.

5. Database Overview

The project uses the AdventureWorks enterprise database containing business information from multiple operational departments.

Major schemas used include:

- Sales
- Production
- Purchasing
- Human Resources
- Person
- Inventory

The database contains information about:

- Customers
- Orders

- Products
- Employees
- Vendors
- Inventory
- Territories

6. Analytics Architecture

The project follows a layered analytics architecture.

Raw Operational Tables

Analytics Base Views

Business Analytics Views

Executive KPI Views

Python Dashboard

This architecture ensures that every dashboard reads only analytical datasets instead of querying raw transactional tables.

7. Business Domains Covered

The analytics layer was developed across multiple business domains.

Business Domain	Description
Sales	Revenue and sales performance
Customers	Customer behavior and segmentation
Products	Product profitability and rankings
Employees	Employee sales performance
Territories	Regional revenue analysis
Inventory	Inventory health monitoring
Vendors	Purchasing and supplier analysis

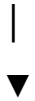
The project satisfies the requirement of using more than five business domains.

8. SQL Pipeline Design

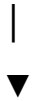
The SQL implementation follows a chained dependency structure.

Operational Tables

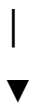
Base Analytics Views



Business Metrics



Customer Segmentation



Regional Analysis

Executive KPIs

Python Visualizations

Each stage reuses previously created views instead of recalculating business metrics.

9. Intermediate Analytics Views

The following reusable analytical views were created.

Base Views

- vw_sales_base
- vw_customer_base
- vw_product_base
- vw_employee_base
- vw_inventory_base
- vw_vendor_base
- vw_territory_base
- vw_date_base
- vw_executive_base

Business Analytics Views

- vw_monthly_revenue
- vw_sales_growth
- vw_customer_segments
- vw_customer_lifetime_value
- vw_customer_retention

- vw_product_performance
- vw_product_profitability
- vw_product_ranking
- vw_employee_performance
- vw_employee_ranking
- vw_regional_revenue
- vw_regional_growth
- vw_inventory_health
- vw_supplier_performance
- vw_purchasing_trends

Executive KPI Views

- vw_best_selling_products
- vw_repeat_customers
- vw_revenue_contribution
- vw_performance_comparison

These analytical views eliminate repeated SQL calculations and simplify dashboard development.

10. Advanced SQL Concepts Implemented

The project demonstrates several advanced SQL techniques.

Multiple Chained CTEs

Used for:

- Customer Retention
- Regional Growth
- Sales Growth

Window Functions

Implemented using:

- RANK()
- DENSE_RANK()
- ROW_NUMBER()
- LAG()

CASE WHEN

Used for:

- Customer Segmentation
- Inventory Classification
- Employee Performance Categories

Conditional Aggregation

Used for:

- Revenue calculations
- Inventory summaries
- Product profitability

Ranking Functions

Used for:

- Product Ranking
- Employee Ranking
- Territory Ranking

Complex JOINs

Multiple business domains were combined using joins involving:

- Sales
- Products
- Customers
- Employees
- Vendors
- Territories

11. Executive KPI Datasets

The analytics layer generates dashboard-ready datasets.

Sales KPIs

- Monthly Revenue
- Sales Growth
- Best Selling Products

Customer KPIs

- Customer Segments
- Customer Lifetime Value
- Repeat Customers
- Customer Retention

Product KPIs

- Product Performance
- Product Profitability
- Product Rankings

Employee KPIs

- Employee Performance
- Revenue Contribution
- Performance Comparison

Territory KPIs

- Regional Revenue
- Regional Growth
- Top Territories

Inventory KPIs

- Inventory Health
- Low Stock Products

Purchasing KPIs

- Supplier Performance
- Purchasing Trends

12. Python Analytics Dashboard

The analytical views were connected to PostgreSQL using SQLAlchemy and Pandas.

The notebook reads only the analytical views.

Visualizations created include:

- Monthly Revenue Trend
- Sales Growth
- Revenue by Territory
- Customer Segmentation
- Product Performance
- Employee Performance
- Inventory Health
- Best Selling Products

Each visualization includes a business explanation describing management insights.

13. Business Insights

Revenue Analysis

Monthly revenue analysis identifies seasonal fluctuations and assists management in planning promotional campaigns during low-performing periods.

Territory Performance

A small number of territories generate the majority of company revenue, indicating opportunities for geographic expansion and targeted marketing.

Customer Behavior

VIP and repeat customers contribute significantly to total revenue, emphasizing the importance of customer retention strategies.

Product Performance

Best-selling products consistently drive business revenue. Maintaining inventory availability for these products is essential.

Employee Performance

Employee rankings highlight performance differences that can support incentive programs and training initiatives.

Inventory Analysis

Inventory health monitoring identifies products approaching critical stock levels before stockouts occur.

14. Executive Recommendations

Business Opportunities

1. Expand investment in top-performing territories.
2. Increase inventory for best-selling products.
3. Improve customer loyalty programs.
4. Promote highly profitable products.
5. Replicate successful sales practices of top-performing employees.

Business Risks

1. Poor-performing territories reduce growth potential.
2. Critical inventory shortages may interrupt sales.
3. Revenue dependency on a limited number of products.
4. Customer retention decline may reduce future revenue.
5. Uneven employee performance impacts sales consistency.

Actionable Recommendations

1. Implement automated inventory monitoring.
2. Develop customer retention campaigns.
3. Increase marketing for profitable products.
4. Review sales KPIs monthly.

5. Conduct regular employee performance evaluations.

15. Challenges Faced

Several implementation challenges were encountered during the project.

- PostgreSQL database import issues due to AdventureWorks CSV formatting.
- Schema compatibility differences between AdventureWorks versions.
- Missing calculated columns such as LineTotal in some tables.
- Data import inconsistencies requiring adjustments to SQL scripts.
- Standardizing analytical view names across the project.
- Resolving connection issues between PostgreSQL and Python.

These challenges were resolved by validating table structures, modifying SQL queries, and creating reusable analytical views.

16. Assumptions

The following assumptions were made during development:

- Operational data was assumed to be accurate after import.
- Missing calculated fields were derived during SQL transformations.
- Analytics views were treated as the single source of truth for reporting.
- Python dashboards accessed only analytical views and never raw transactional tables.
- Business metrics were calculated using available transactional data.

17. Future Improvements

Future enhancements may include:

- Materialized views for improved performance.
- Automated ETL pipelines.
- Incremental data refresh.
- Power BI or Tableau integration.

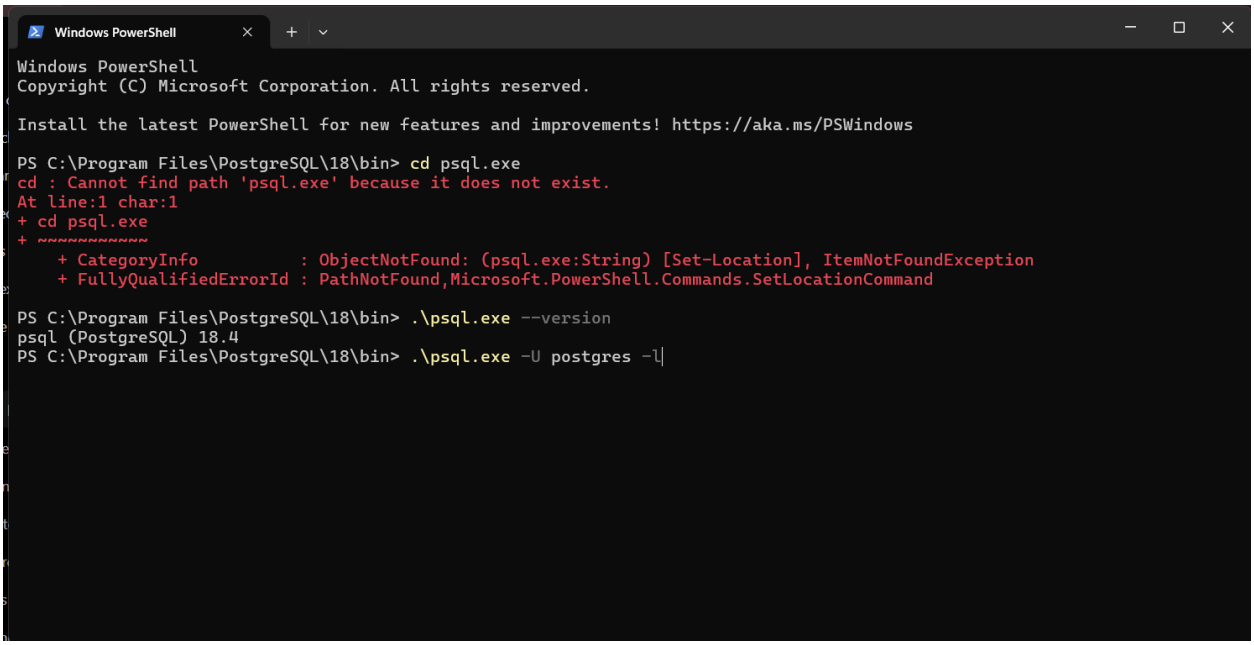
- Machine learning-based sales forecasting.
- Customer churn prediction models.
- Inventory demand forecasting.
- Interactive web dashboards using Streamlit or Dash.

18. Conclusion

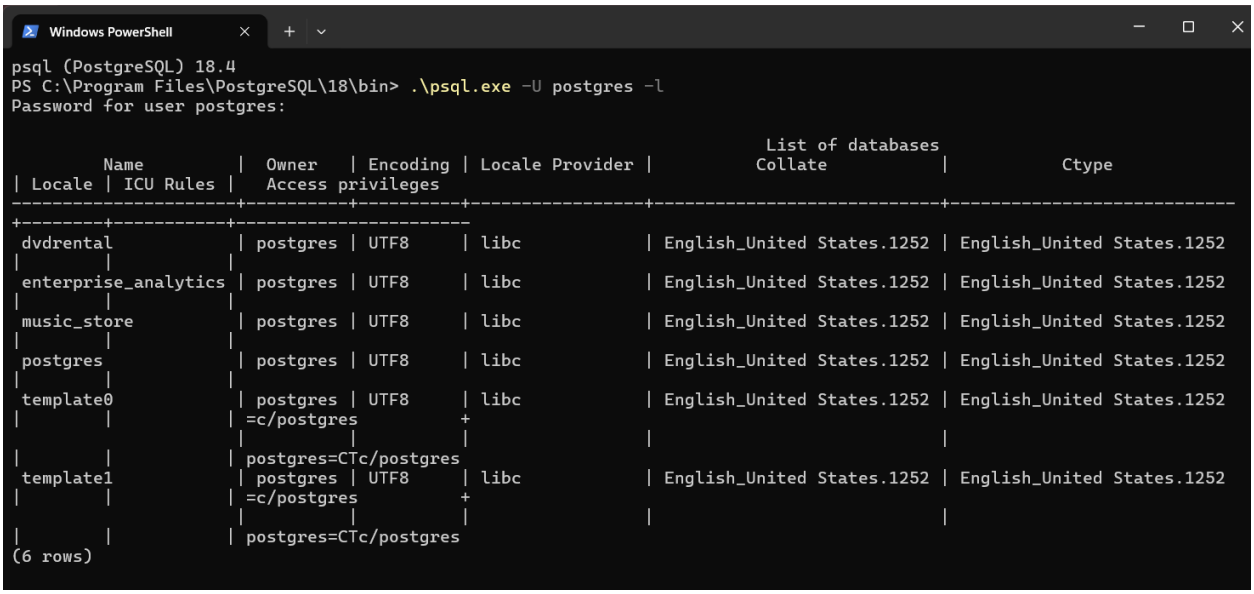
This project successfully demonstrates the implementation of a reusable enterprise analytics layer using PostgreSQL and Python.

Instead of repeatedly querying normalized operational tables, reusable analytical views were created to centralize business logic and simplify reporting. The SQL pipeline supports future dashboard development while maintaining scalability and maintainability.

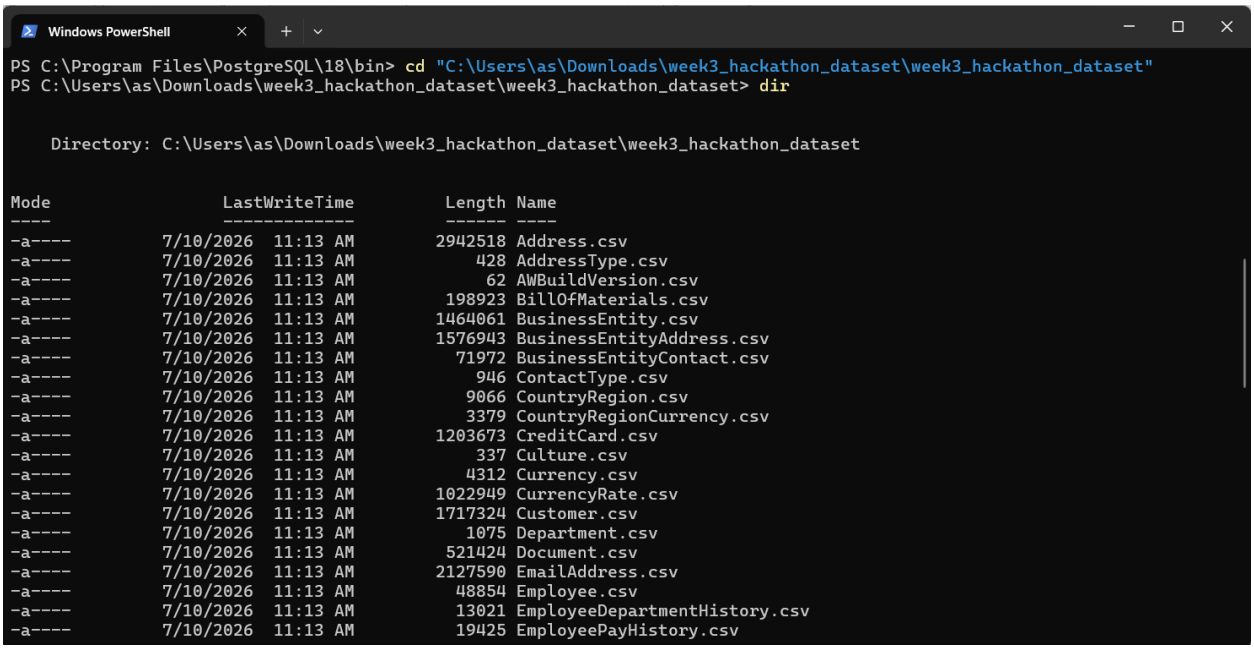
The Python notebook consumes only analytical views to generate executive dashboards that provide valuable business insights. The overall solution follows Analytics Engineering principles and establishes a robust foundation for future business intelligence and reporting initiatives.



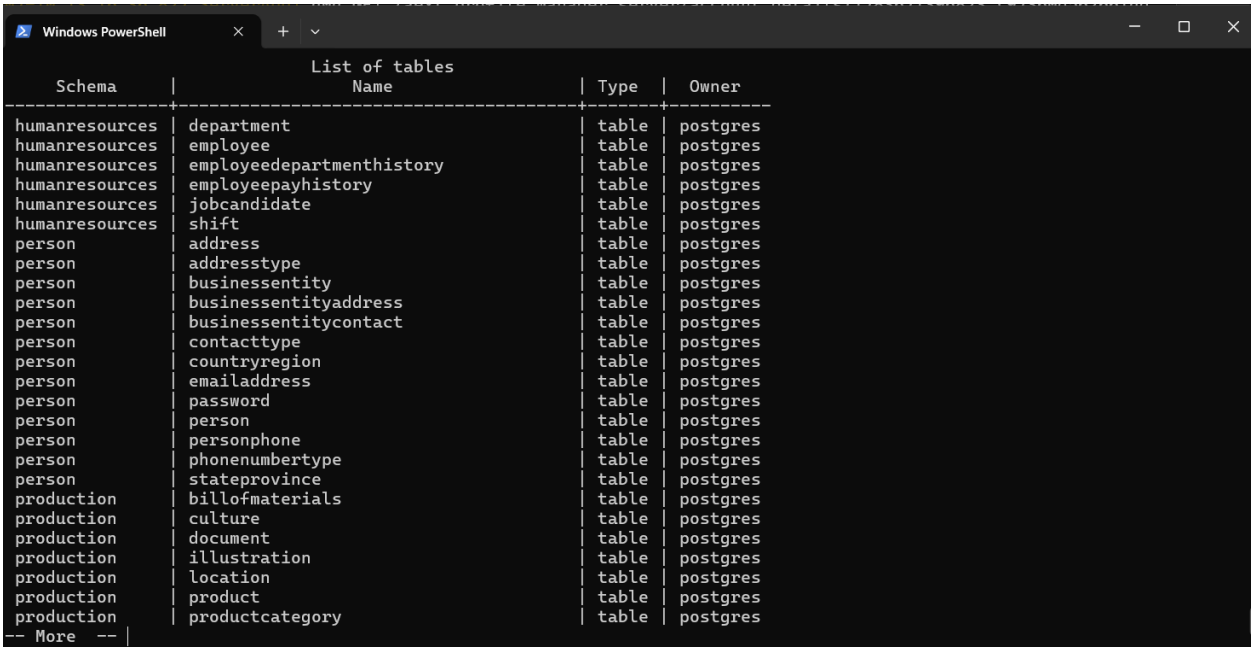
Database creation



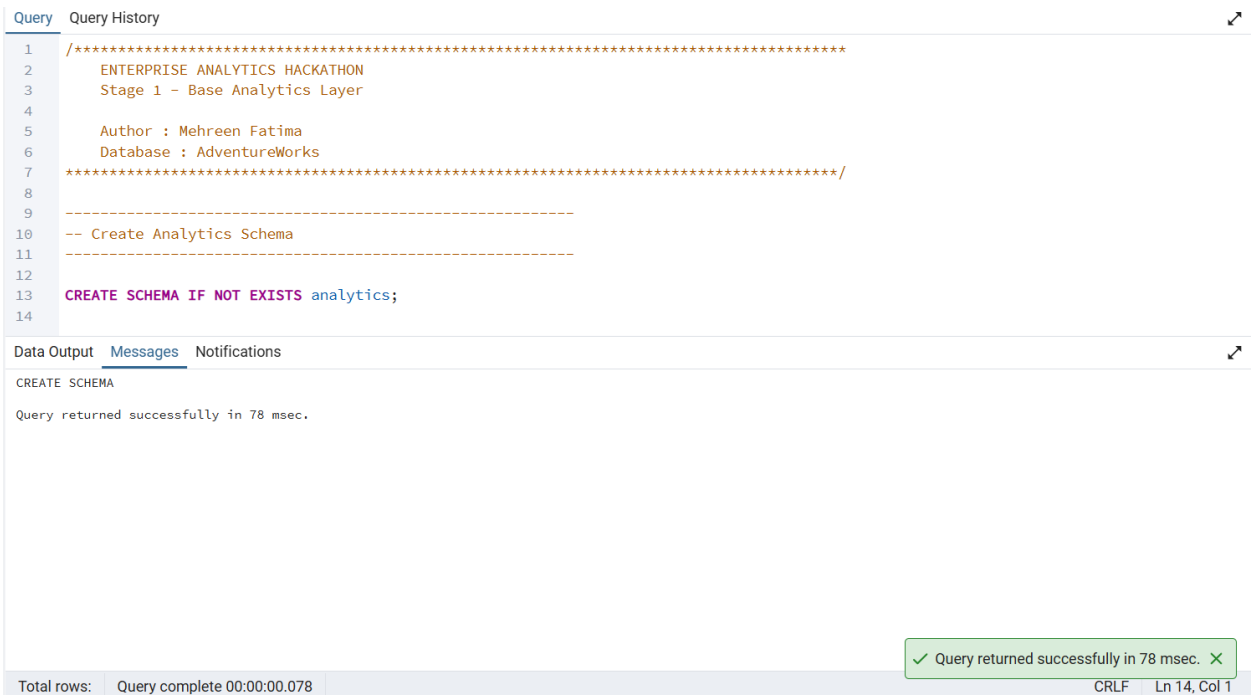
Reference files:



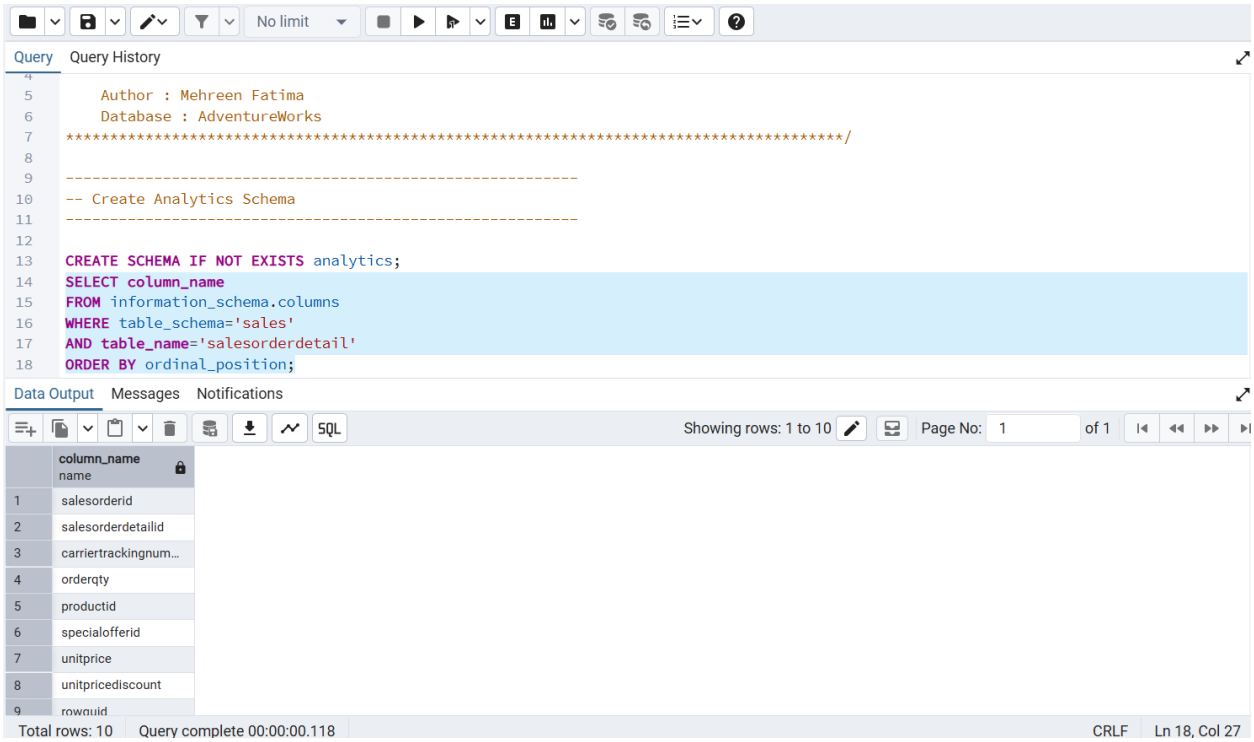
Imported tables



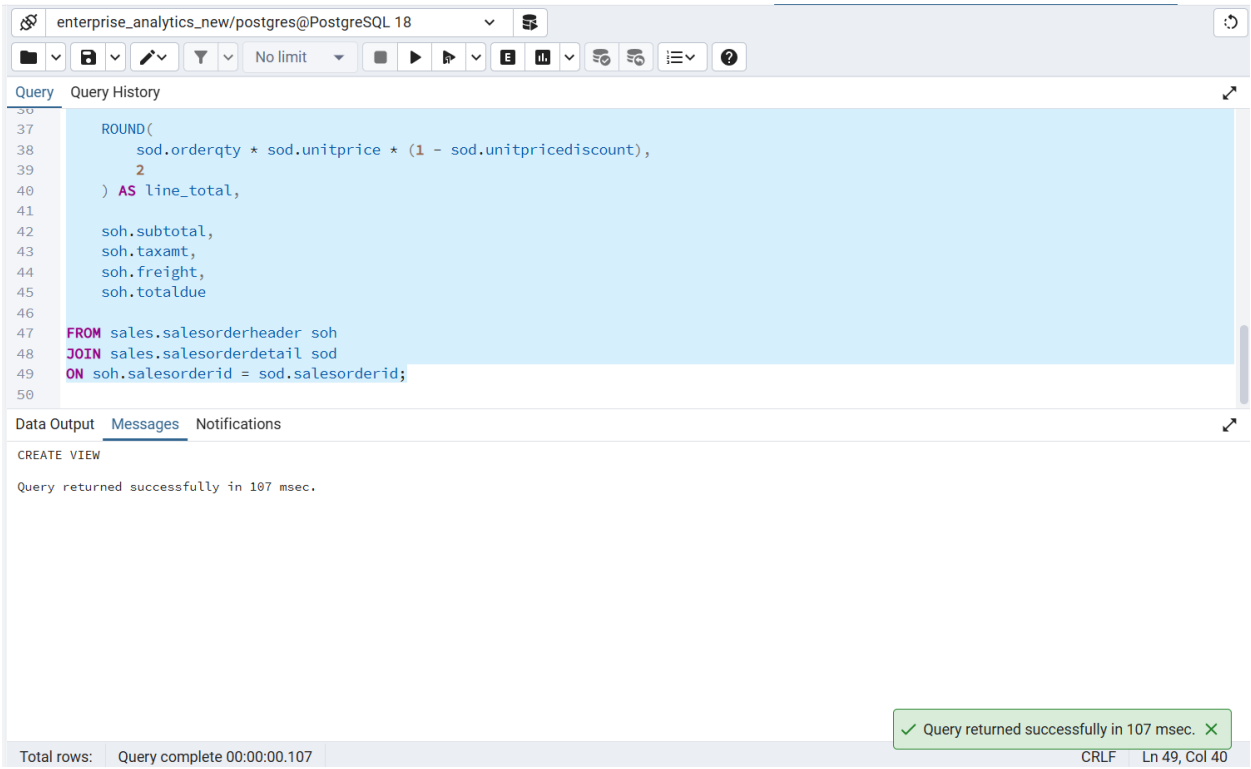
Schema Creation



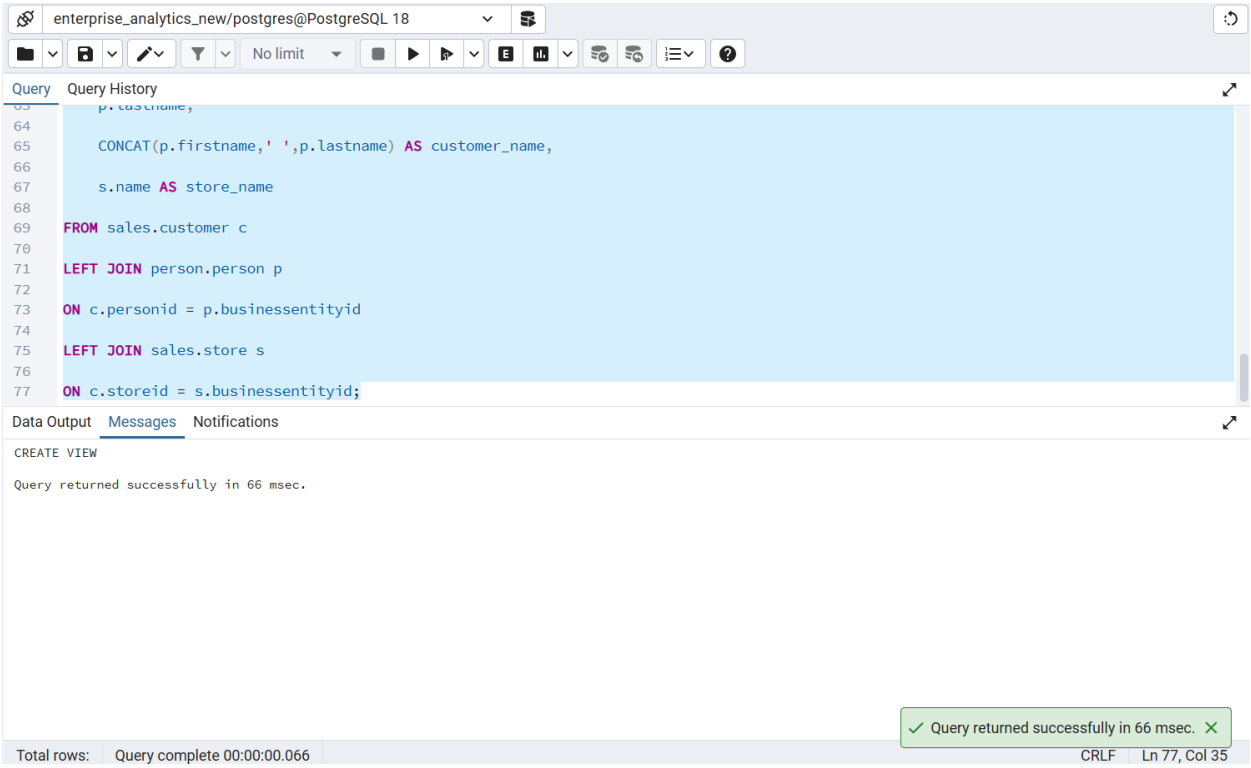
Column Names:



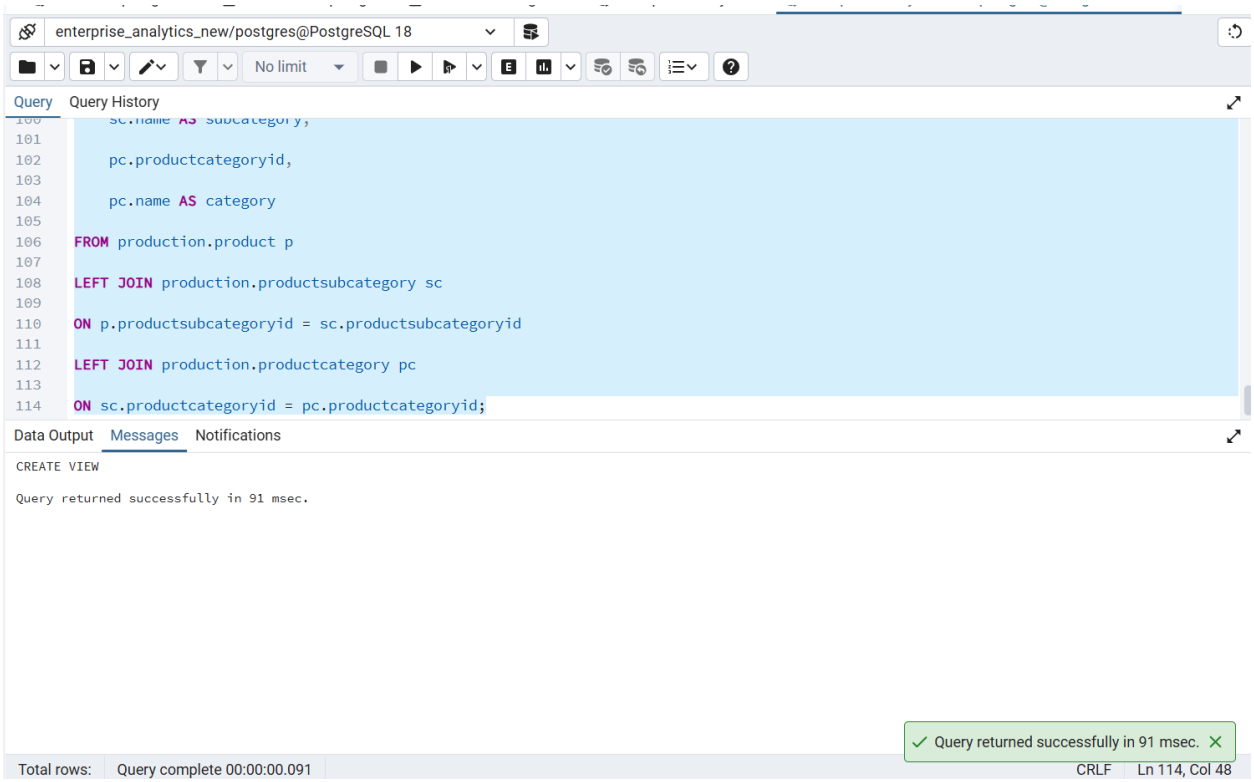
Sales Base View:



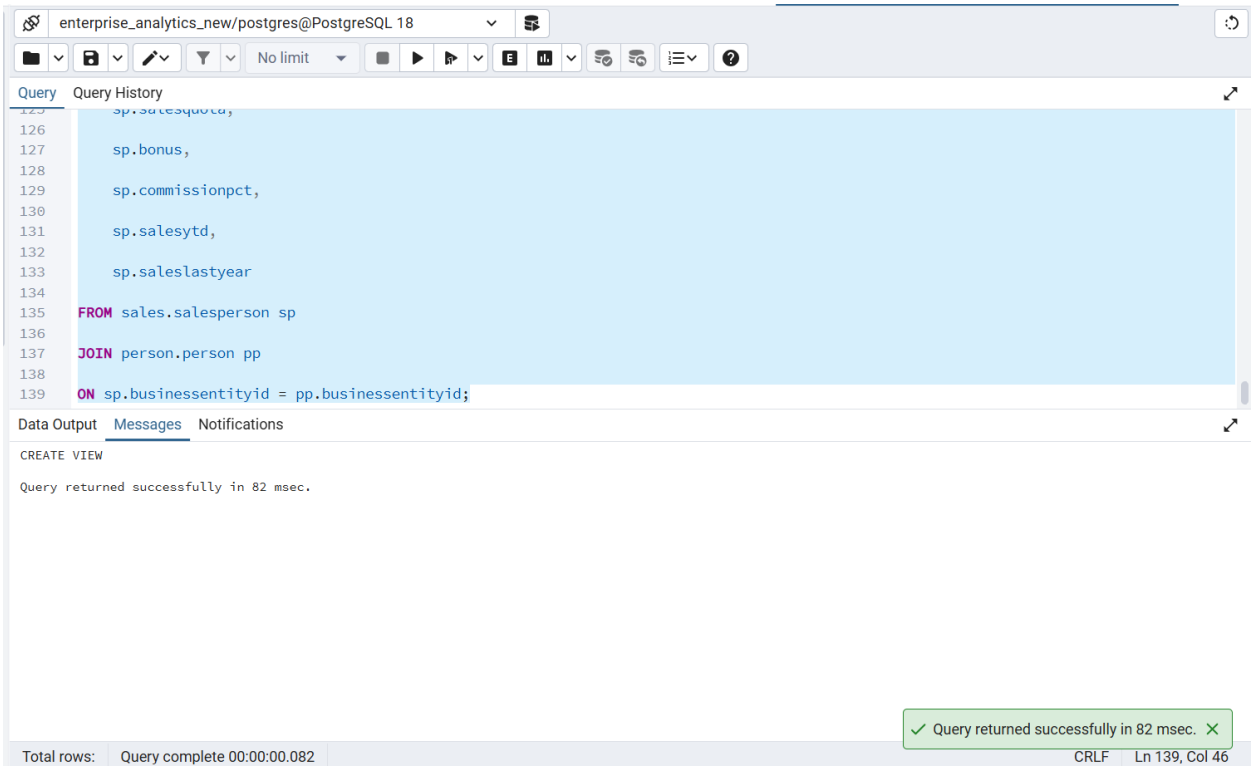
Customer Base view:



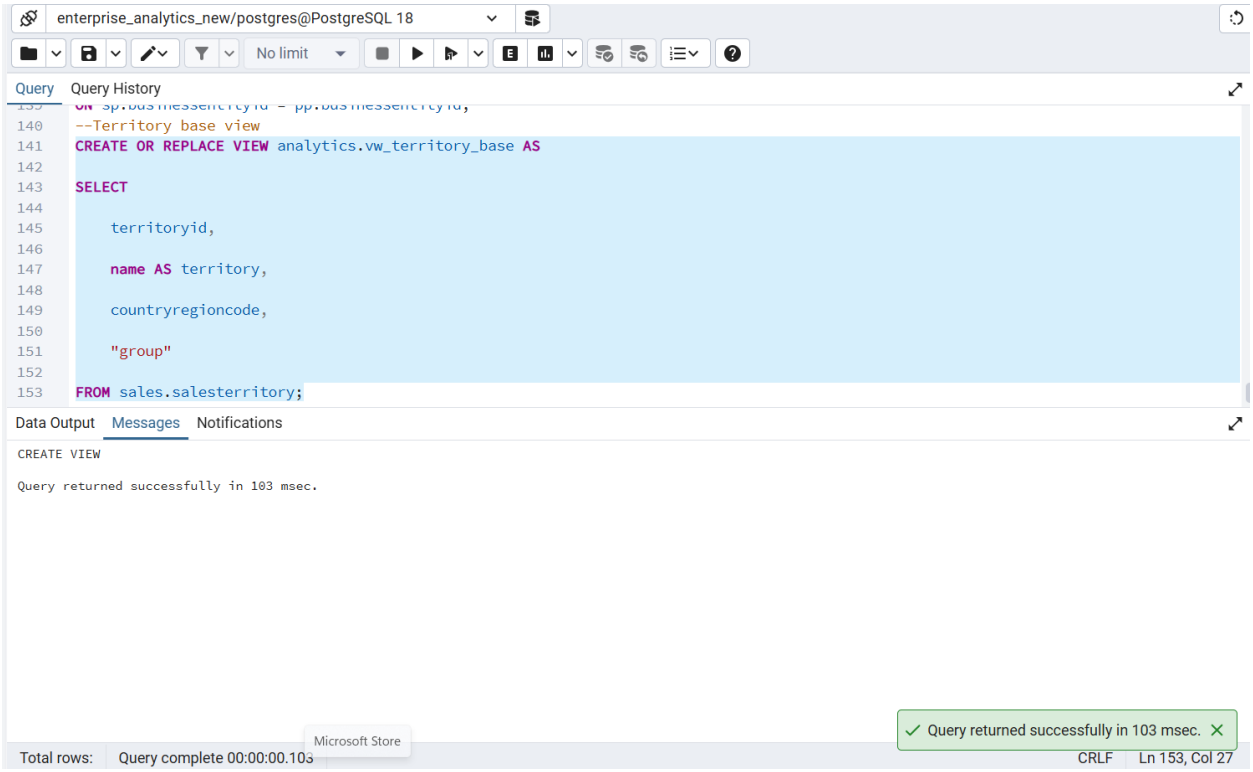
Product Base view



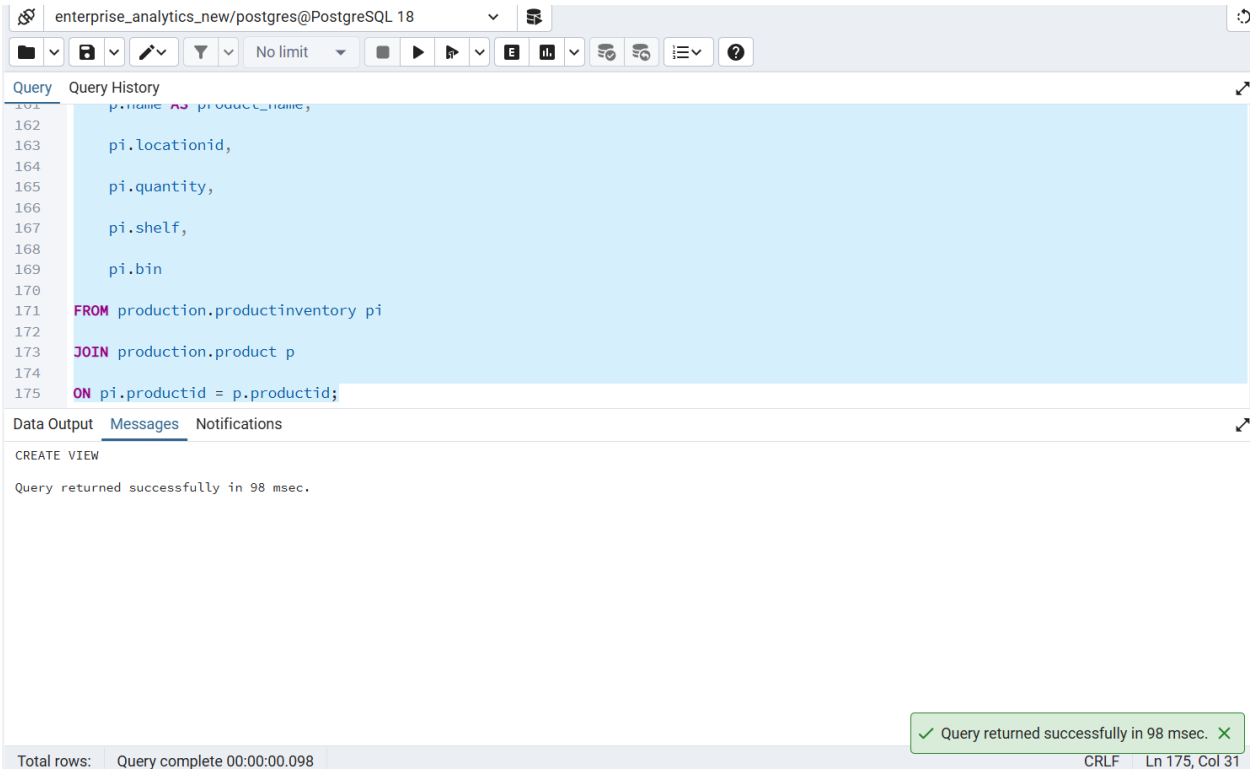
Employee Base View



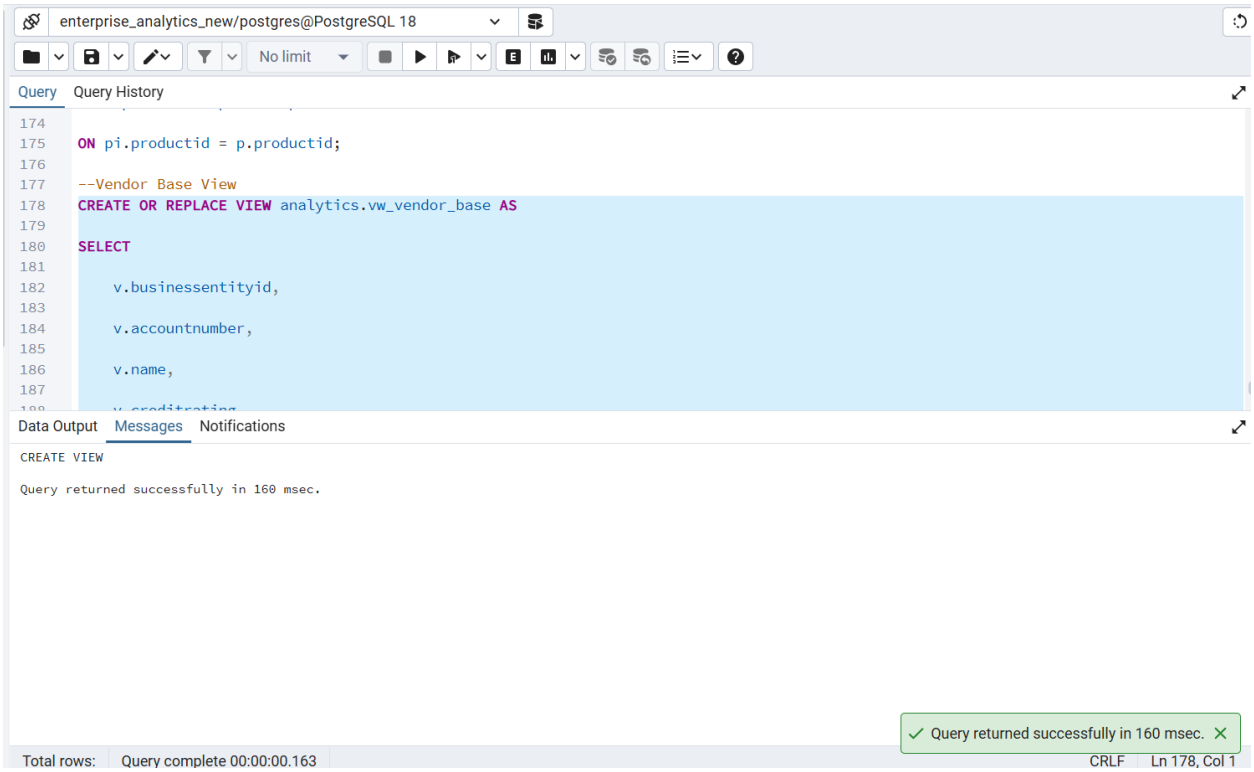
Territory Base View



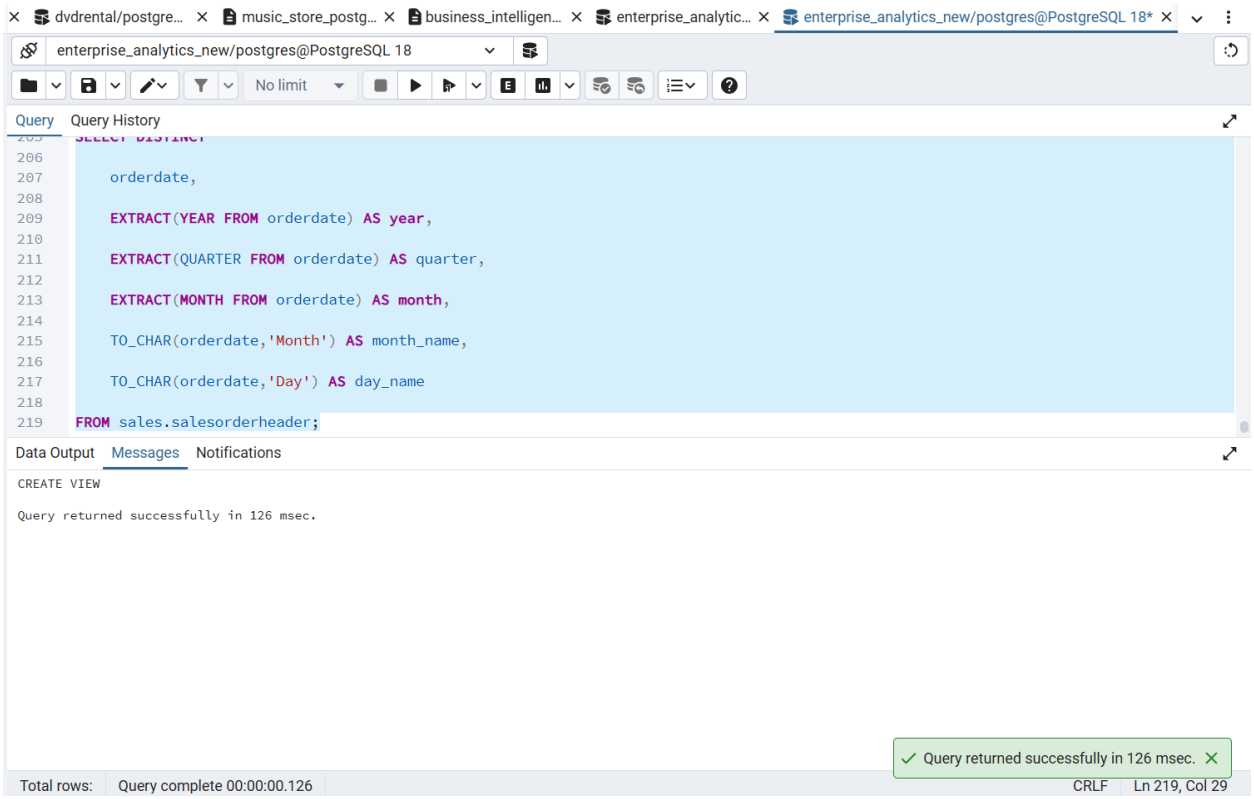
Inventory Base View



Vendor Base View

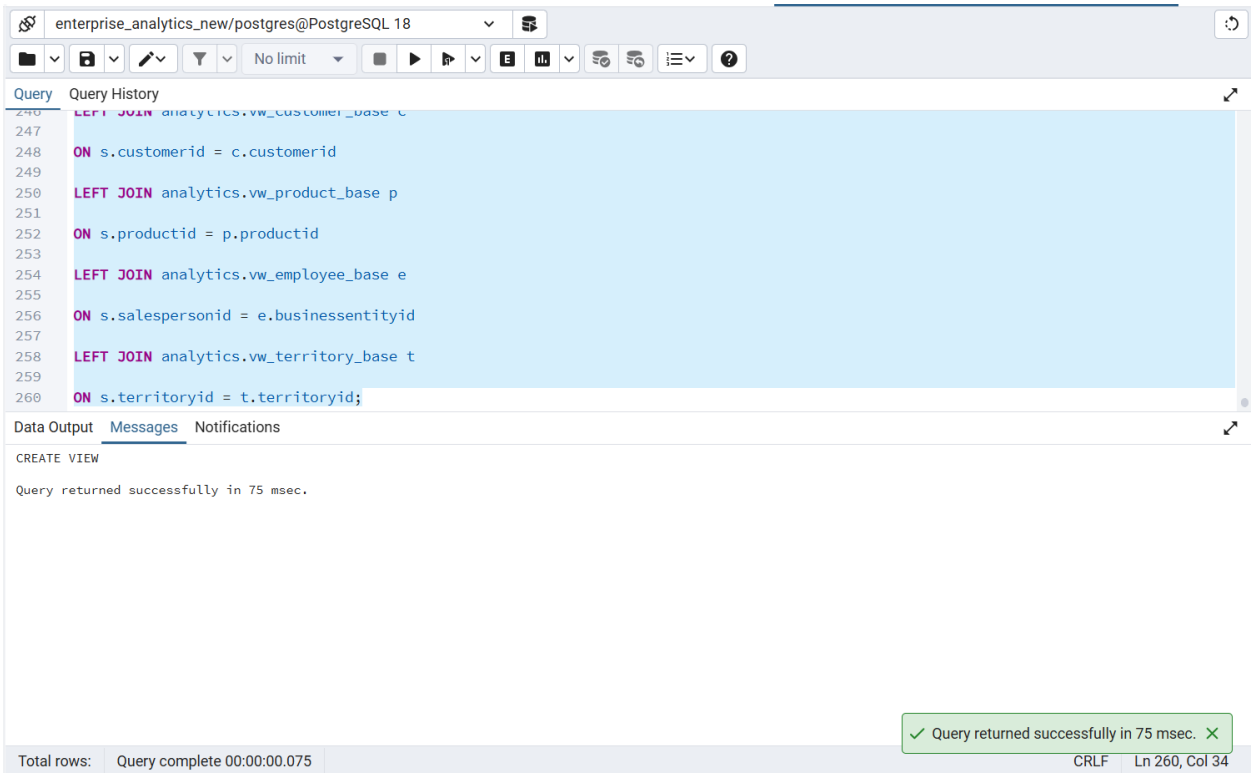


Data Analytics Base View:

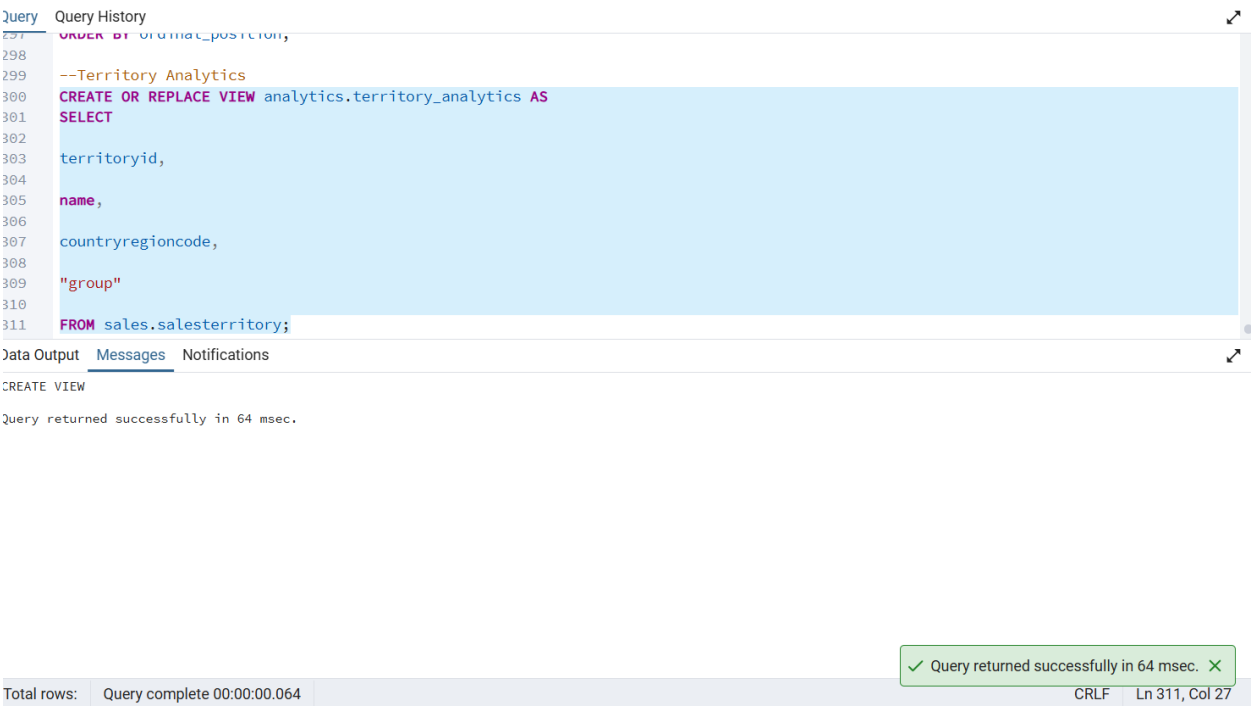


Executive Base View

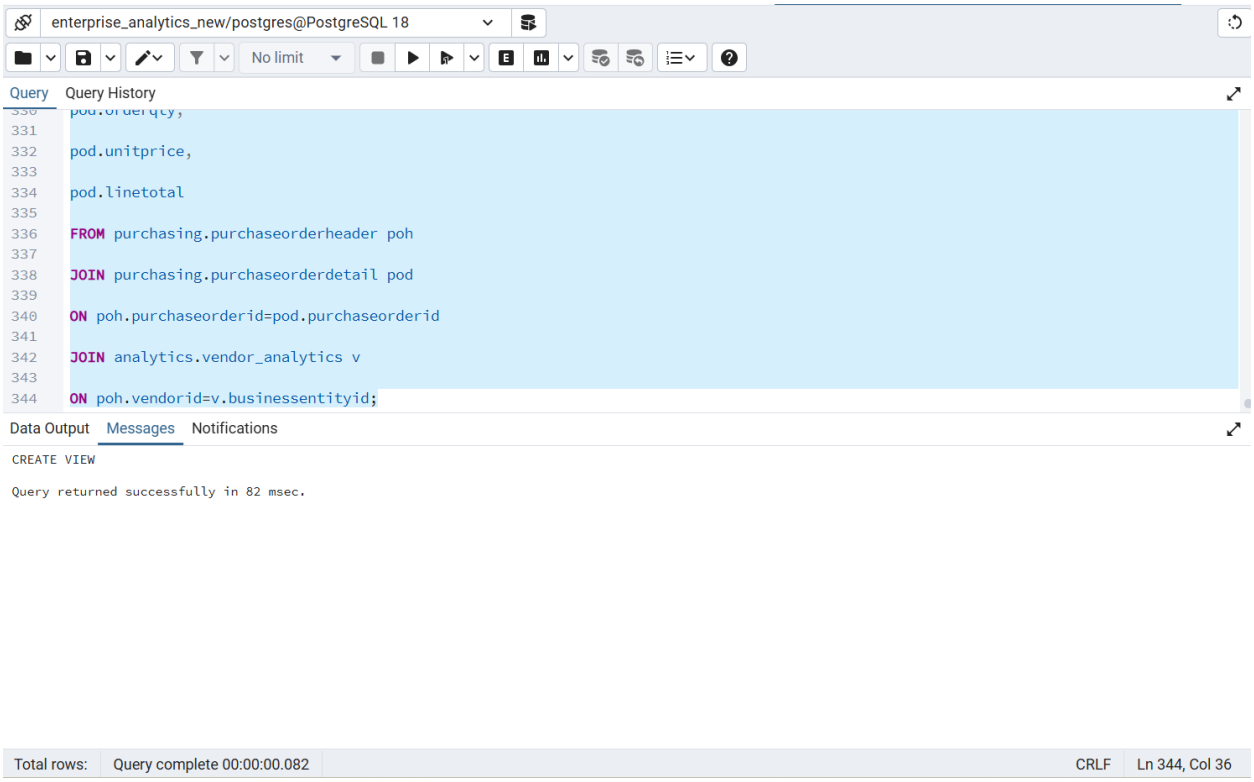
This is the first reusable business layer.



Territory Analytics:



Purchasing Base Analytics:



Task 02:

Dependency Chain

Raw Tables

↓

Task 1 Analytics Views

↓

Business Metrics

↓

Customer Segmentation

↓

Regional Analysis

↓

Executive KPI Views

↓

Python Dashboard

This satisfies your supervisor's requirement.